



SharePie

AI Settlement Agent

정산이 먼저, 결제는 그 다음.

조건은 말로, 계산은 코드로, 돈은 에스프로, 기록은 블록체인에.

말로 정한 분담 조건을 AI가 해석하고, 코드가 계산·검증한 몫만 참여자가 직접 에스프로에 예치합니다.
보류 기간이 지나면 결제자에게 지급되고, 조건과 결과는 Ethereum Sepolia 테스트넷에 남습니다.



대회 GWDC Korea Hackathon 2026 · FuriosaAI × Bricksum · Agent Finance Bonus Track, Challenge A

AI Kiln API (NPU 추론) · qwen3-32b

체인 Ethereum Sepolia 테스트넷 · ShareLedger / PieToken

Team SharePie

선언한 기능은 하나, AI Settlement Agent

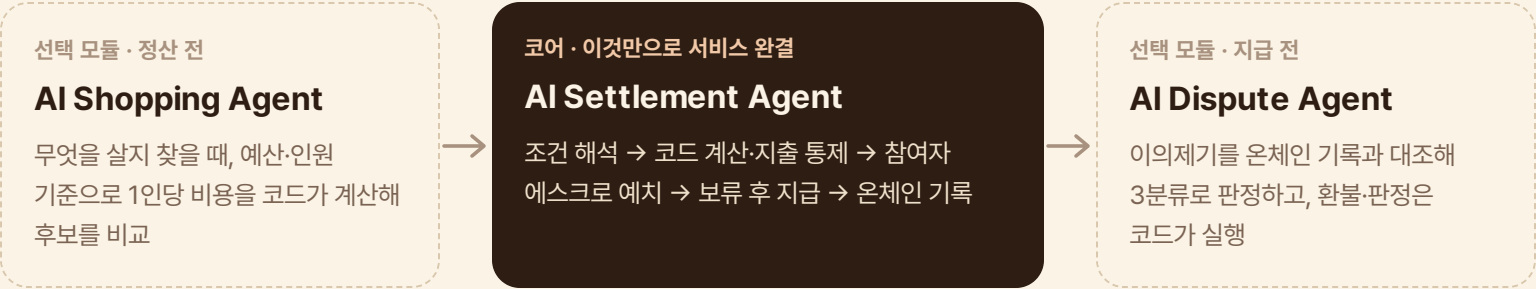


평가는 README에 선언한 이 한 문장을 기준으로 받습니다. 쇼핑과 분쟁은 정산 코어에 꽂았다 뺄 수 있는 선택 모듈입니다.

README 한 문장 선언

SharePie is, at its core, an AI Settlement Agent that interprets users' natural-language cost-sharing conditions, calculates and verifies a compliant settlement, locks the approved amounts in escrow, releases them after the hold period, and records the result on-chain. Two optional modules extend it: an AI Shopping Agent for pre-settlement discovery, and an AI Dispute Agent for post-settlement investigation.

사용자가 말로 정한 부담 조건과 지출 한도(1인·총 한도, 허용 판매처)를 AI가 해석하고, 코드가 금액을 계산·검증해 규칙을 통과한 경우에만 참여자가 MetaMask로 직접 Sepolia 에스크로(ShareLedger)에 테스트 토큰 PieCoin을 예치하며, 위반 시 결제 없이 중단 기록(Blocked)이, 이의제기 시 AI 판정과 환불 기록(DisputeResolved·Refunded)이 온체인에 남는다.



두 모듈이 없어도 "참여자·금액·조건 입력 → 정산"은 끝까지 동작합니다. 브리프 예시 중 조건을 지키는 구매 에이전트(지출 통제)와 증거·분쟁 도구의 성격은 정산 흐름 안에 포함했습니다.

심사 기준으로 보는 페이지

	User Need & Workflow 사용자·문제, 입력부터 결과까지	3-4
	Kiln API Integration & Efficiency 응답 → 행동, 흐름별 토큰, 에너지	5-6
	Blockchain Integration 체인 상태, E2E 트랜잭션	7
	Selected Function 한 문장 선언 (이 페이지)	2
	Condition Checks & Evidence 조건 변경 실행, 제3자 검증, 분쟁	8-9
	한계와 재현 방법 데모 구조의 한계, 검증 명령	10

돈을 함께 쓰면, 누군가는 총무가 된다

대상 사용자는 친구·모임·공동구매에서 돈을 함께 쓰는 사람들입니다. 대학생·직장인 모임과 동네 공동구매가 중심입니다.

조건이 쉬이면 손계산이 틀린다

“진주는 5천원 적게, 술값은 안 마신 사람 빼고” 같은 조건을 사람이 계산하면 1원 단위에서 어긋나고 합계가 맞지 않습니다.

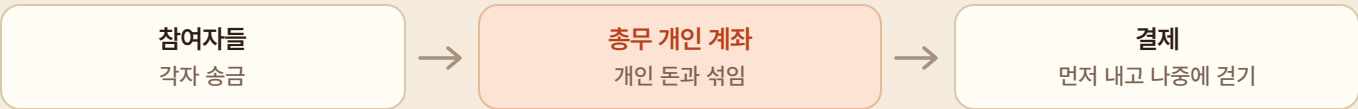
근거가 남지 않는다

누가 냈는지, 왜 그렇게 나눴는지 기록이 없으니 시간이 지나면 다툼이 됩니다.

총무를 믿는 것 말고는 방법이 없다

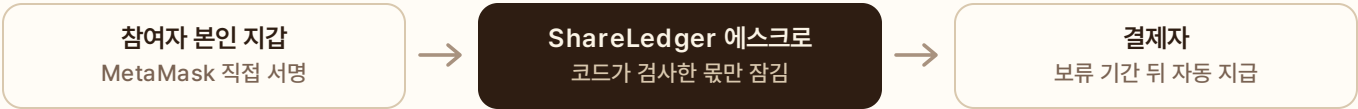
한 사람이 돈을 모아 대신 결제하는 구조에서는, 그 사람을 믿는 것 외에 확인할 방법이 없습니다.

지금의 정산



근거는 단톡방 대화뿐이고, 잔액은 총무만 압니다.

SharePie



조건 해시·분담표·지급 기록이 온체인에 남아, 누구든 검증할 수 있습니다.

현실에서 반복되는 일 (언론 보도 사례)

학생회비 횡령	간부가 공금을 개인 계좌에 보관하며 사적으로 사용 — 인하대 학생회, 2023년, 7,600만원
공동구매 선입금 먹튀	굿즈 공동구매로 125명에게 입금받고 물건을 보내지 않음 — 2026년, 총 2,090만원
불투명한 회비 관리	“얼마 남았어요?”에 답하지 않는 동호회·동창회 총무 — 범죄가 아니어도 매번 생기는 갈등

실제 사용자 — 베타 실사용 (2026-09-29 기준)

6명	92턴	41건	11건
베타 사용자	Pie와의 대화	답변 평가 도움 됐어요 28 아쉬워요 13	정산 지급 6 · 중단 3 · 진행 2

베타 기간 정산은 모의 체인 모드였고, 실체인 모드 데이터를 추가 수집하고 있습니다. 대화는 제공에 동의한 사람만 비식별 저장합니다.

말 한 줄이 정산 인증서가 되기까지

이해와 판단은 AI가, 숫자·규칙·체인 호출은 코드가 맡습니다. AI 응답은 코드가 검증한 뒤에만 실행됩니다.

	참여자	AI	코드	체인	누가 처리하는지 표시
1	참여자				그룹방에 조건을 말한다 금액·사람·조건을 한 문장으로. 대화 중에 조건을 바뀌어도 된다
2	AI				Stage 1 조건 해석 <code>settlement.analyze</code> 자연어 → 규칙 JSON. 모호하면 한 번에 한 가지만 되묻는다 ("조금 더'가 정확히 얼마인가요?")
3			코드		Stage 2 계산과 지출 통제 <code>settlement.calculate</code> <code>.policy</code> 1원 단위 분담 · 합계=총액 · 1인/총 한도 · 허용 판매처 · 체인 가용 잔액 · 한글 금액 오판싱 가드. AI 호출 0회
4	AI				Stage 3 결과 설명 <code>settlement.explain</code> 결과를 쉬운 말로 설명하고, 앱은 확인 카드(총액·사람별 금액·근거)를 띄운다
5			코드	체인	정산 등록 <code>createSettlement</code> 분담표·조건 해시·목적을 기록. 지출 통제 위반이면 등록 대신 <code>blockSettlement</code> 로 중단 기록
6	참여자			체인	본인 지갑으로 예치 <code>approve</code> → <code>lockForSettlement</code> 각자 MetaMask로 서명. 잔액이 부족하면 컨트랙트가 거부해 일부만 잠기는 상태가 없다
7			코드	체인	보류 후 지급 <code>releaseToRecipient</code> 전원 예치 뒤 이의제기 기간(베타 180초)이 지나면 결제자에게 지급, 정산 인증서 확정
8	AI		코드		지급 전 이의제기가 오면 코드가 기록을 대조하고 AI가 3분류로 판정, 환불·판정 실행은 코드 (9쪽)

입력 — Run 1 실제 조건

“진주2는 5천원 적게 내고 나머지 세 명이 나눠줘”

삼겹살 공동구매 총 35,900원 · 예산 40,000원 · 결제자 진주2

AI Stage 1 출력 (규칙 JSON 형식)

```
{"status": "ok",
"rule": {"adjustments": [
  {"name": "진주2", "kind": "less",
"value": 5000}]}}
```

코드 Stage 2 결과

5,225	10,225	10,225	10,225
진주2	진우2	민재2	지현2

✓ 합계 35,900 = 총액 ✓ 예산 40,000 이내 ✓ 판매처 허용

체인 사용 가능한 결과물 — 정산 인증서

지급 TxHash [0x932d...d7c6](#) · 조건 원문 · 분담표 · Etherscan 링크
조건이 대화 중 바뀌면 Stage 1만 다시 부르고, Stage 2는 같은
함수를 그대로 재실행합니다.

Kiln 응답이 에이전트의 다음 행동을 정한다

모든 Kiln 호출은 단계 태그와 입력·출력 토큰, 지연시간을 남기고, 응답마다 “무엇을 하기로 했는지” 결정 기록이 따라붙습니다.

모델

Kiln API (NPU) · qwen3-32b

KILN_MODEL 환경변수로만 참조

엔드포인트

api.bricksum.com/v1

/chat/completions

응답 형식

JSON 모드 고정 (KILN_TOOL_MODE=json) —

tool calling 인자가 빈 값으로 오는 것을 실측하고 고정

호출 단계	Kiln 응답	그 응답으로 코드가 하는 일
settlement.analyze	status: ok + 규칙 JSON	Stage 2 계산·지출 통제 → 통과하면 createSettlement , 위반이면 blockSettlement (AI 재호출 없음)
settlement.analyze	status: need_info	한 가지만 되묻기. 체인 호출 없음
settlement.explain	설명 문장	확인 카드에 표시. 숫자는 코드가 만든 카드 값만 인용
dispute.investigate	GENUINE_ERROR, refund=all	refundParticipant × N → resolveDispute(2) → Refunded
dispute.investigate	NORMAL_APPROVAL · BAD_FAITH	resolveDispute(1 3) → 결제자에게 지급
assistant.step	도구 선택 (예: 메뉴 가격 검색)	코드가 도구를 실행하고 가격 합계·예산·인분을 검증 (assistant.tool , 0 토큰)

실제 기록 — Sepolia, 2026-09-28 22:35 UTC

이의제기 “판매자가 품질로 주문을 취소했어요” → Kiln 판정

GENUINE_ERROR → 환불 0xfa90...f228 · 0x7171...5b57 ·

0xfb6d...6500 → 판정 0x9a21...55a3

AI 응답을 그대로 믿지 않는 코드 가드

근거 없는 금액 → 되묻기 · 검증 안 된 메뉴 조합 → 조합 검사 강제 · 기록

불일치가 있으면 판정을 GENUINE_ERROR로 보정 · 본인 서명 기록 없이는

BAD_FAITH 판정 불가

다양하게

교촌 허니콤보 1개 + BBQ 황금올리브반반 + 황금알치즈볼

51,440원 +배송 3,000원

1인 예상
약 13,610원

구성
4인분

남는 예산
5,560원

이 상품 선택하기



네 명이 6만원 안에서 치킨을 시키는 조합을 세 가지 추천드려요.

1. 가성비: BBQ 황금올리브치킨 + BHC 뿌링클, 1인 10,663원 (남는 예산 17,350원)

2. 푸짐하게: 교촌 허니콤보웨이감자세트, 1인 7,350원 (남는 예산 30,600원)

3. 다양하게: 교촌 허니콤보 + BBQ 황금올리브반반, 1인 13,610원 (남는 예산 5,560원)

가격은 네이버쇼핑 기준이에요. 조합을 정하면 앱에서 정산방을 만들어 주세요.

Kiln qwen3-32b 3회 · 11,060 토큰 · 8.4초

도움 됐어요

아쉬워요

실제 앱 화면 (베타) — 답변마다 모델·호출 수·토큰·지연이 표시됩니다(주황 표시). 카드의 1인 예상·남는 예산은 코드가 계산한 값이고, AI는 인용만 합니다.

필요한 곳에만 추론한다

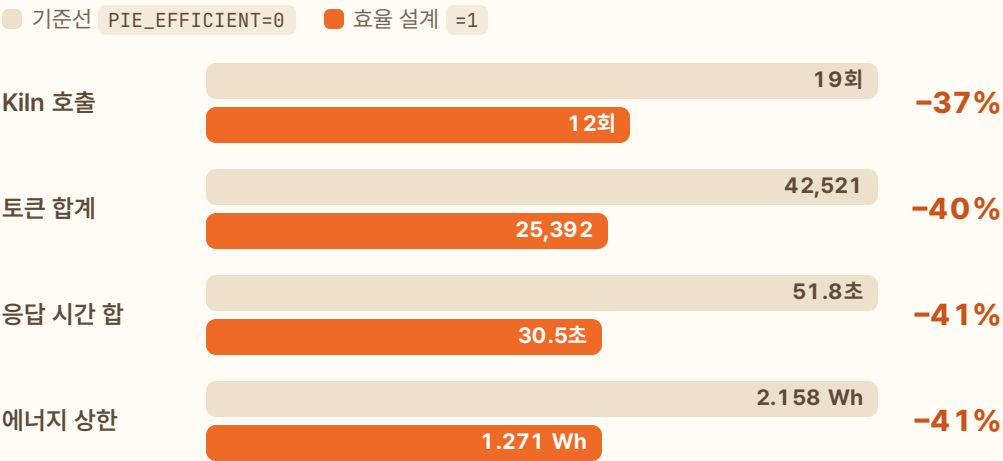
토큰은 합계가 아니라 흐름별로 보고합니다. 계산·검증·지출 통제·검색·체인 호출은 처음부터 코드라서 0 토큰, 0 Wh입니다.

흐름별 AI 토큰 — 실측 (v39 서버 누적, 2026-09-29 07:36 KST, qwen3-32b)

흐름	단계	처리	Kiln 호출	코드 처리	토큰	에너지 상한
정산 코어	Stage 1 해석 <code>analyze</code>	AI	48	4	74,853	5.633 Wh
	Stage 2 계산 <code>calculate</code>	코드	0	45	0	0
	지출 통제 <code>policy</code>	코드	0	26	0	0
	Stage 3 설명 <code>explain</code>	AI	2	0	658	0.146 Wh
분쟁·증거	기록 대조 <code>dispute.records</code>	코드	0	5	0	0
	판정 <code>dispute.investigate</code>	AI	5	0	5,282	1.036 Wh
공동구매·추천	검색 · 1인당 비용 계산	코드	0	11	0	0
	조건 해석 · 비교 설명	AI	8	0	8,083	1.028 Wh
Pie 대화	의도 파악 · 도구 실행	코드	0	37	0	0
	에이전트 판단 <code>assistant.step</code>	AI	22	0	63,376	2.019 Wh
합계			85	128	152,252	≈ 9.9 Wh

에너지 추정 = 실측 Kiln 응답 시간(초) × 150 W × 카드 1장 ÷ 3600. 150 W는 FuriosaAI RNGD의 TDP이고, 그 시간 동안 카드가 이 요청만 처리했다고 보는 상한값입니다(배치 공유 미반영). 실측 전력 데이터는 없어서 토큰 수와 호출 수를 대리 지표로 씁니다. 예: 조건 해석 1회 2.8초 → 약 0.12 Wh.

같은 11개 장면, 기준선 vs 효율 설계 — 실측 (2026-09-30 02:13)



11개 장면 모두 결과 동일 — 분담 금액, 되물기, 한도 경고, 추천 카드가 기준선과 같습니다.

- E2

금액·인원만 있는 단순 나누기는 코드가 해석 (AI 0회)
- E3

똑같이 나눈 결과는 문장 틀로 설명 (AI 0회)
- E4

그룹방에서 이름만 부르면 코드가 안내 (AI 0회)
- E1

요청 종류에 필요한 도구 설명만 전송
- E5·E6

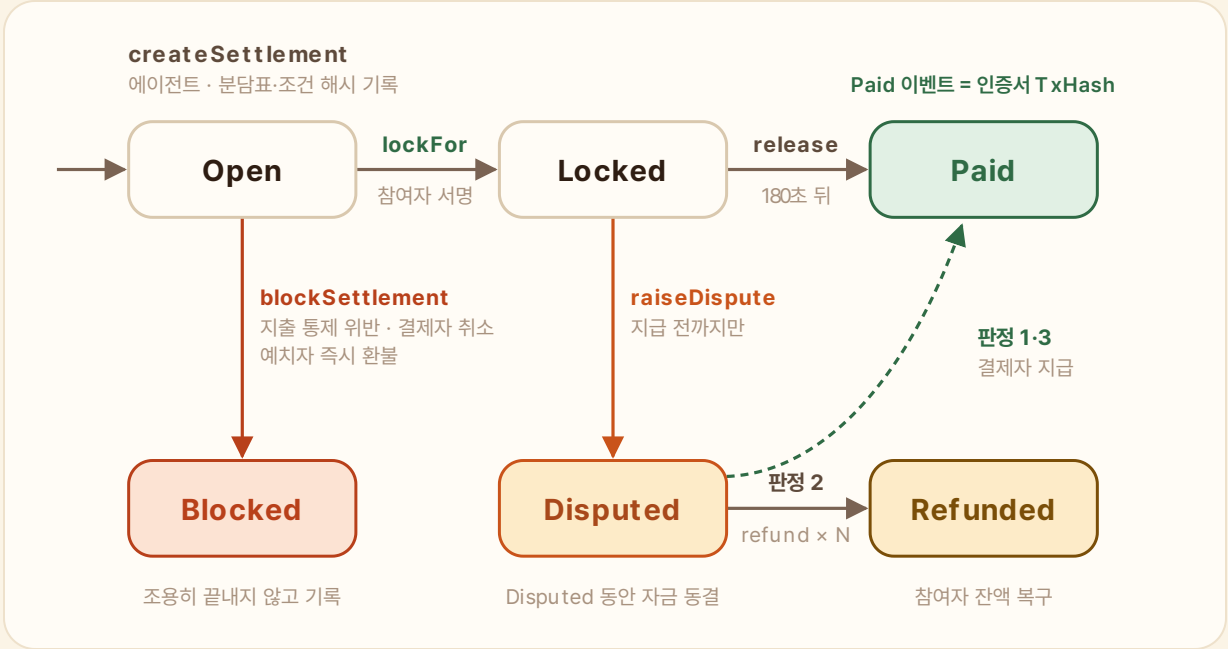
예시·지식·대화 기록을 필요한 만큼만
- E7

`/no_think` — 긴 사고는 분쟁 판정·구매 계획만

조건 해석이 필요한 장면(여러 조건·퍼센트·모호·지출 한도)은 정확도를 위해 줄이지 않았습니다. 재현:
`python tests/token_bench.py`

체인이 읽고, 쓰고, 정산하는 것

정산 상태는 ShareLedger 컨트랙트가 들고 있고, 에이전트는 5초마다 그 상태를 읽어 다음 호출을 정합니다.



돈을 잠그는 서명은 항상 참여자 본인

MetaMask로 `approve` → `lockForSettlement`.
컨트랙트가 `msg.sender` 의 토큰만 옮기므로
에이전트는 참여자 지갑에서 돈을 뺄 수 없습니다.

에이전트 지갑이 서명하는 것

충전·등록·중단·판정·환불·지급·현금 결제 확인. AI는 이
함수들을 직접 부르지 못하고, 코드가 검증한 뒤
호출합니다.

배포 — Ethereum Sepolia (chainId 11155111)

ShareLedger `0xF297240957c3aB10458Dc1A4C6eC2eA18292529E`

PieToken `0xF5cB871A8890bd1D34bf36E749F012D95589E0fD`

에이전트 지갑 `0x33f446E980bd8F1CeeAEFA43B2FfDB5e45c46C36`

배포 블록 11801645 · PIE는 decimals 0, 1 PIE = 1원 표시, 실화폐 가치 없음

쓰기 `createSettlement` (분담표, conditionHash, 목적), `blockSettlement`, `raiseDispute`,
`resolveDispute`

정산 `lockForSettlement` (참여자 서명), `releaseToRecipient`, `refundParticipant`

읽기 `getSettlement`, `getMembers`, `releaseAt`, `escrowOf` → 화면 자금추적과 5초 감시 루프

End-to-end 실행 1회 (Run 1, 실제 Sepolia 2026-09-28)

등록 `0xb5d5...60c0` → 참여자 예치 3건 → 지급·인증서 `0x932d...d7c6`

매칭 로그 `docs/evidence/sepolia-2026-09-28T22-36-17/summary.md` · `run1.json`

실측 가스: 등록 513,574 · 예치 128,725 · 지급 146,673 · 차단 51,378 · 이익제기 39,426 · 환불 61,997 · 판정 65,226. 정산 1건(등록+지급) ≈ 0.0014 ETH. 컨트랙트 테스트 30개 통과.

조건을 바꾸면 코드가 멈추고, 멈춘 사실도 남는다

같은 요청을 조건만 바꿔 다시 실행했습니다. 범위를 벗어나면 코드가 멈추고, 결제 없이 `BLOCKED` 를 온체인에 남깁니다.

같은 요청 “진주2는 5천원 적게 내고 나머지 세 명이 나눠줘” 총 35,900원 → 코드 계산 $5,225 / 10,225 \times 3$

Run 1 · 정상

Paid

예산 40,000원

✓ $35,900 \leq 40,000$ · 판매처 허용

등록 → 참여자 3명이 본인 지갑으로 예치 → 180초 보류 →
결제자에게 지급, 정산 인증서 발급

등록 `0xb5d5...60c0`

지급 `0x932d...d7c6`

Run 1.5 ① · 1인 한도 축소

Blocked

1인 한도 9,000원

✗ $10,225 > 9,000 \rightarrow$ `OVER_PERSON_CAP`

등록·예치·지급 0건, AI 호출 0회. 에이전트가
`blockSettlement` 로 중단 사유를 온체인에 기록

중단 `0x4e77...f5cb`

Run 1.5 ② · 총 한도 축소

Blocked

총 한도 30,000원

✗ $35,900 > 30,000 \rightarrow$ `OVER_TOTAL_CAP`

등록·예치·지급 0건, AI 호출 0회. 누가 봐도 왜 멈췄는지 알 수
있게 사유와 함께 기록

중단 `0x8367...33fc`

Run 1.5 ③ · 허용 판매처 위반 `MERCHANT_NOT_ALLOWED`

“쿠팡만 허용”인데 네이버스토어 결제 → 등록 없이 중단. 증거 실행기에 포함했고 TxHash는 다음
증거 실행에서 기록합니다. 베타 실사용에서는 실제 사용자 조건으로 중단 3건(판매처 위반 2, 잔액
부족 1)이 나왔습니다(모의 체인 모드).

AI가 우회할 수 없는 안전장치

잔액이 모자란 예치는 컨트랙트가 revert해 일부만 잠기는 상태가 없음 · 합계=총액 · 체인 가용 잔액
검사 · 한글 금액 오판싱 가드(문장에 ‘만/억’이 있는데 총액이 더 작으면 `AMOUNT_SUSPECT` 로 거부) ·
불법 목적은 AI 호출 전에 코드가 차단

기록만 보고도 판단할 수 있다

누구나 Etherscan 기록만으로 정산이 조건대로였는지 확인할 수 있고, 이의제기는 기록 대조 → AI 판정 → 코드 실행으로 처리합니다.

제3자 검증 — 기록만으로 조건 준수를 판단하는 4단계

- 1 정산 인증서에서 **조건 원문과 분담표**를 본다 (앱 '정산 인증서 보기' 또는 summary.md)
- 2 Etherscan에서 등록 트랜잭션의 `SettlementCreated` 이벤트를 열어 `conditionHash` · `shares[]` · `purpose` 를 읽는다
- 3 조건 원문을 같은 방식으로 해시한다 (keccak256, 코드 `chain.condition_hash`) — 값이 같으면 이 온체인 정산은 그 조건으로 만들어진 것이다
- 4 `Locked` · `Paid` · `Blocked` · `Refunded` 이벤트로 누가 얼마를 예치했고, 어디로 지급·환불됐는지(또는 왜 멈췄는지) 확인한다

keccak256(조건 원문)
인증서의 문장을 직접 해시

=

conditionHash
SettlementCreated 이벤트의 값

체인에는 실명 대신 지갑 주소만 올라갑니다. 목적 문자열은 코드가 이름·연락처를 지운 뒤 60자 이내로 기록합니다.

Run 2 — 이의제기에서 환불까지 (실제 Sepolia, 별도 정산)

- 참여자** 전원 예치 완료 → 상태 `Locked` (지금 뒤에는 이의제기가 불가능해 별도 정산으로 실행)
- 참여자** 이의제기 “판매자가 품질로 주문을 취소했어요” → `raiseDispute` `0x156e...8d31`, 자금 동결
- 코드** 기록 대조 `dispute.records` (0 토큰) 최초 요청 → Stage 1-2 결과 → 등록 → 누가 본인 서명으로 예치했는지 → 지급 상태
- AI** `dispute.investigate` → **GENUINE_ERROR**, refund=all
- 코드** 가드 확인 후 `refundParticipant` × 3 `0xfa90...f228` · `0x7171...5b57` · `0xfb6d...6500`
- 코드** `resolveDispute(2)` `0x9a21...55a3` → `Refunded`, 참여자 잔액 전액 복구

1 NORMAL_APPROVAL

정상 승인 → 이의 기각, 결제자 지급

2 GENUINE_ERROR

진짜 착오 → 환불 후 판정 기록

3 BAD_FAITH_DISPUTE

악의적 이의 → 근거 공개, 결제자 지급

판정은 이 세 가지로 고정합니다. 기록 불일치가 있으면 GENUINE_ERROR로 보정하고, 본인 서명 기록 없이는 BAD_FAITH 판정을 내릴 수 없으며, 코드의 기록 대조 없이 AI 판정만으로 환불하지 않습니다.

정직하게 밝히는 한계와 직접 확인하는 방법

데모용으로 단순화한 부분을 숨기지 않았습니다. 아래 명령과 링크로 같은 결과를 직접 재현할 수 있습니다.

에이전트 지갑 하나가 여러 역할을 서명

충전·등록·중단·판정·환불·지급은 데모용으로 에이전트 지갑 하나가 서명합니다. 참여자 예치만 본인 MetaMask 서명입니다.

Run 2의 착오는 시연용 시나리오

“판매자 품질 취소”는 주입한 상황입니다. 판정은 AI가, 환불 실행은 코드가 했습니다.

베타 정산은 모의 체인 모드

2026-09-29 베타 정산 11건은 모의 체인 기록이라 심사 증거로 쓰지 않았고, 실체인 모드로 추가 수집 중입니다.

증거 실행기는 테스트 지갑으로 서명

`npm run evidence` 는 참여자 예치를 참여자 테스트 지갑으로 스크립트가 서명합니다(같은 컨트랙트 호출). 실제 MetaMask 예치 기록은 베타 실체인 데이터로 보강합니다.

에너지는 상한 추정

실측 전력이 아니라 실측 지연 × 150 W로 계산한 상한입니다. 배치 처리를 반영하면 요청당 에너지는 더 작습니다.

테스트넷 전용

PieCoin은 실화폐 가치가 없고 실제 계좌·카드·메인넷은 다루지 않습니다. 배포 스크립트는 메인넷 chainId를 거부합니다.

```
# 컨트랙트 · 체인 증거 (hardhat/)
npm test                # 테스트 30개
npm run smoke           # 서버·권한·해석·가드·체인 쓰기
npm run evidence        # Run 1 / 1.5 / 2 → summary.md
# AI 토큰 · 에너지
GET /api/usage/report.md
python tests/token_bench.py
```

컨트랙트 (Etherscan)

sepolia.etherscan.io/address/0xF297...529E

체인 실행 증거

`docs/evidence/sepolia-2026-09-28T22-36-17/`

토큰 효율 실측

`docs/evidence/token_efficiency_20260930-0213.md`

규칙 · 설계 문서

`README.md · docs/TOKEN-EFFICIENCY.md · docs/API.md`



정산이 먼저, 결제는 그 다음.

SharePie, 미래를 나누고 공유하세요

Team SharePie · GWDC 2026 Challenge A